

SÉANCE 1

Systemes d'exploitation Windows & Linux

Introduction générale aux systèmes d'exploitation

Ouafae EL AISSAOUI

ouafae.elaiassaoui@usmba.ac.ma





Qu'est-ce qu'un système d'exploitation ?

Système d'exploitation (*noté SE, ou OS pour Operating System*)

- Un ordinateur se compose d'une partie matérielle (hardware) et d'une partie logicielle (software).
- Le système d'exploitation est un logiciel qui s'installe après le formatage d'un PC.
- Il est chargé d'assurer la liaison entre les ressources matérielles, l'utilisateur et les applications (traitement de texte, jeux vidéo, navigateur...).





Composants d'un ordinateur



Processeur (CPU)

Central Processing Unit — Unité Centrale de Traitement. Manipule les informations numériques (données binaires) et exécute les instructions stockées en mémoire.



Mémoire vive (RAM)

Random Access Memory. Stocke temporairement les données et programmes que l'ordinateur est en train d'utiliser.



Interfaces d'entrée-sortie

Souris, clavier, écran, imprimante... permettent l'interaction entre l'utilisateur et la machine.



Mémoire externe

Disque dur (HDD), SSD (Solid State Drive), lecteur CD-ROM... stockage permanent des données.



Le processeur (CPU) : trois unités clés

Le processeur peut être découpé en trois grandes unités qui coopèrent à chaque instruction :

Unité de contrôle (UC)

Dirige l'exécution des instructions. Indique à la RAM, aux registres et à l'ALU ce qu'ils doivent faire.

ALU — Unité arithmétique et logique

Effectue les calculs mathématiques (addition, soustraction...) et les comparaisons logiques (ex. : $A > B$?).

Les registres

Petites mémoires ultra-rapides à l'intérieur du processeur, pour stocker temporairement des données ou instructions.

Processeur (CPU) — schéma de fonctionnement

Unité de contrôle

fetch → decode → dispatch

ALU

opérations arithm. & logiques

Registres

mémoire ultra-rapide

Cache processeur

L1 / L2 : mémoire rapide



La mémoire cache

Mémoire cache : mémoire très rapide, proche du processeur, qui garde les données et instructions utilisées souvent, pour un accès plus rapide que depuis la RAM.

1 Première demande

- Le processeur demande une donnée.
- Absente du cache → cache miss : récupérée depuis la RAM, puis copiée dans le cache.

2 Demande suivante

- Même donnée redemandée → trouvée directement dans le cache : cache hit.
- Accès beaucoup plus rapide que depuis la RAM.

Hiérarchie des niveaux de cache



Plus on s'éloigne du CPU : plus la capacité augmente, plus la vitesse diminue.



LE PROCESSEUR

Les registres

Petites mémoires ultra-rapides à l'intérieur du processeur, qui stockent temporairement les données et instructions en cours de traitement.



Registre d'instructions (IR)

Instruction Register

Contient l'instruction en cours d'exécution.

Exemple : « additionner 2 et 5 ».



Compteur ordinal (PC)

Program Counter

Indique l'adresse de la prochaine instruction à exécuter en mémoire.

Permet au processeur de savoir où aller chercher la suite du programme.



Registre accumulateur (ACC)

Accumulator

Sert à stocker les résultats intermédiaires des opérations arithmétiques/logiques.

Exemple : pour $2 + 5$, l'ALU dépose 7 dans l'accumulateur.



Les types de mémoires : synthèse

Une hiérarchie de compromis : plus une mémoire est rapide, plus sa capacité est réduite.

Type	Vitesse	Capacité	Rôle
Registres	Très rapide	Très petite	Stocker les valeurs immédiates pour le CPU
Cache (L1, L2, L3)	Rapide	Moyenne	Stocker les données / instructions fréquemment utilisées
RAM	Moyenne	Grande	Stocker les données du programme en cours
Disque dur / SSD	Lent	Très grande	Stocker les fichiers et programmes

↑ Vitesse

Registres

Cache (L1/L2/L3)

RAM

Disque dur / SSD



Les appels système

Des fonctions spéciales fournies par le noyau du système d'exploitation qui permettent à un programme utilisateur (application) de demander un service au noyau.



Lire ou écrire un fichier



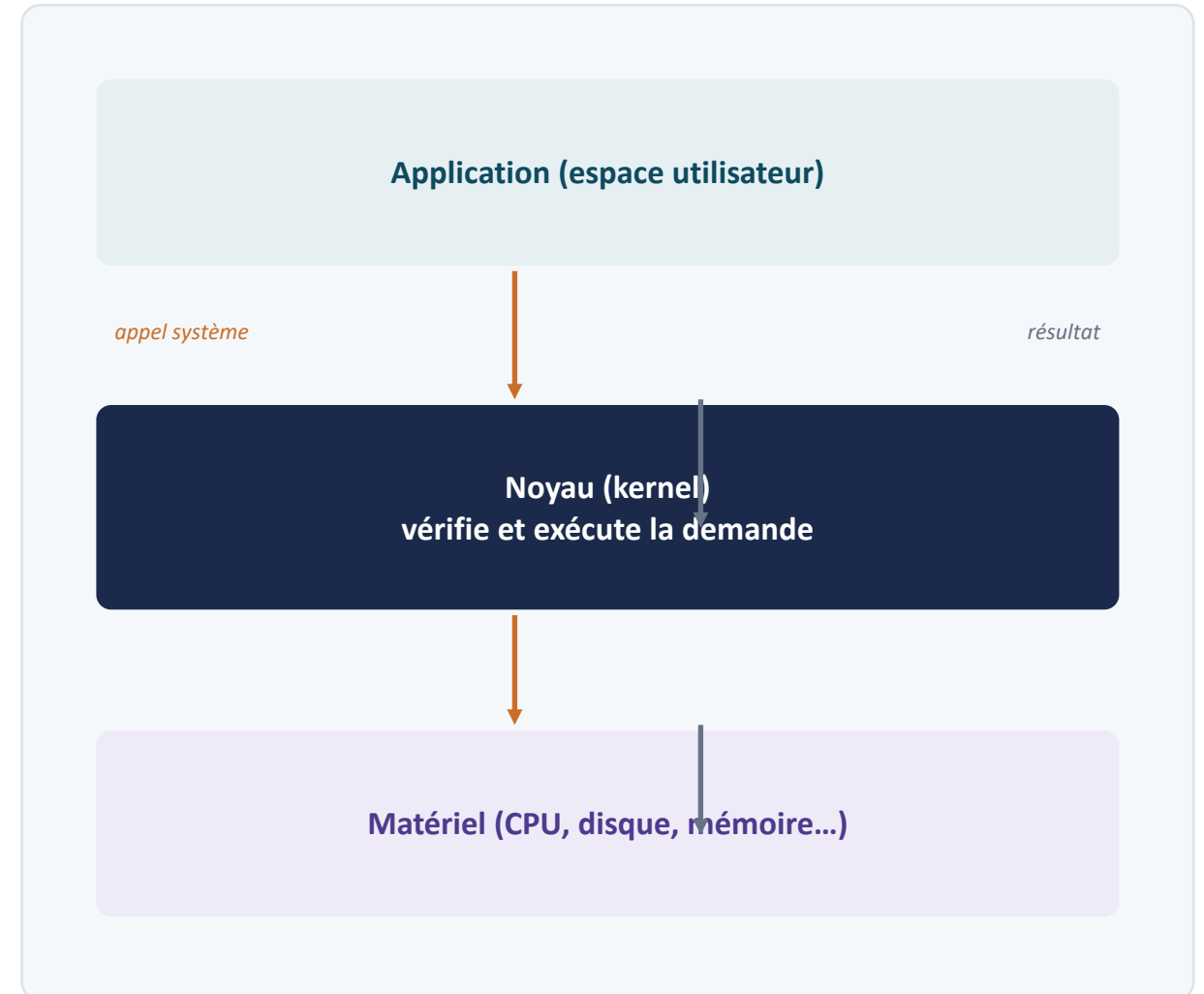
Créer ou terminer un processus



Allouer de la mémoire



Communiquer avec des périphériques





Lire ou écrire un fichier (Linux, C)

code.c

```
#include <stdio.h>
#include <fcntl.h>      // pour open()
#include <unistd.h>     // pour write(), close()

int main() {
    int fd;
    const char *texte = "\nNouvelle ligne ajoutée avec C !";

    // Ouvrir le fichier en mode écriture + ajout (append)
    fd = open("notes.txt", O_WRONLY | O_APPEND);
    if (fd == -1) {
        perror("Erreur lors de l'ouverture du fichier");
        return 1;
    }

    // Écrire à la fin du fichier
    if (write(fd, texte, 32) == -1) {
        perror("Erreur lors de l'écriture");
        close(fd);
        return 1;
    }

    printf("Ligne ajoutée avec succès !\n");

    // Fermer le fichier
    close(fd);
    return 0;
}
```

Appel système	Rôle
open()	Ouvre le fichier dans le mode spécifié.
write()	Écrit une chaîne dans le fichier.
close()	Ferme le fichier pour libérer les ressources.

Terminal

```
gcc code.c -o code
./code
```

À retenir

Ce n'est pas le programme qui écrit lui-même sur le disque : c'est le noyau qui exécute réellement l'opération physique, via l'appel système.



Créer un processus (fork, Linux)

code.c

```
#include <stdio.h>
#include <unistd.h> // pour fork(), getpid(), getppid(), sleep()

int main() {
    pid_t pid;
    pid = fork();

    if (pid == 0) {
        // Code du fils
        printf("Je suis le processus Fils. PID = %d, PPID = %d\n",
               getpid(), getppid());
        sleep(20);
    } else {
        // Code du parent
        printf("Je suis le processus Parent. PID = %d, PID du Fils = %d\n",
               getpid(), pid);
    }
}
```

Arborescence des processus

```
init (PID 1)
├── bash (Terminal 1)
│   ├── ls
│   └── top
└── bash (Terminal 2)
    └── firefox
```

Terminal

```
gcc code.c -o code
ps -ef | grep proc
```

Appel système	Rôle	Processus
fork()	Crée un nouveau processus fils en dupliquant le parent.	Parent → fils
getpid()	Obtient le PID du processus courant.	Parent et fils
getppid()	Obtient le PID du processus parent.	Surtout le fils
sleep()	Suspend l'exécution pendant un temps donné.	Parent et fils
write() (via printf)	Écrit les messages sur le terminal.	Parent et fils

Chaque processus Linux est créé par un autre processus, appelé processus parent : init (PID 1) est l'ancêtre de tous les autres.



SYNTHÈSE

Scénario : lancer la calculatrice et faire $2 + 5 =$

1

Clic sur l'icône Calculatrice → événement capté par un périphérique d'entrée (souris/clavier) → transmis à l'OS via son pilote (driver).

2

L'OS utilise des appels système (open/read) pour ouvrir le fichier exécutable, et le charge depuis le disque (HDD/SSD) vers la RAM.

3

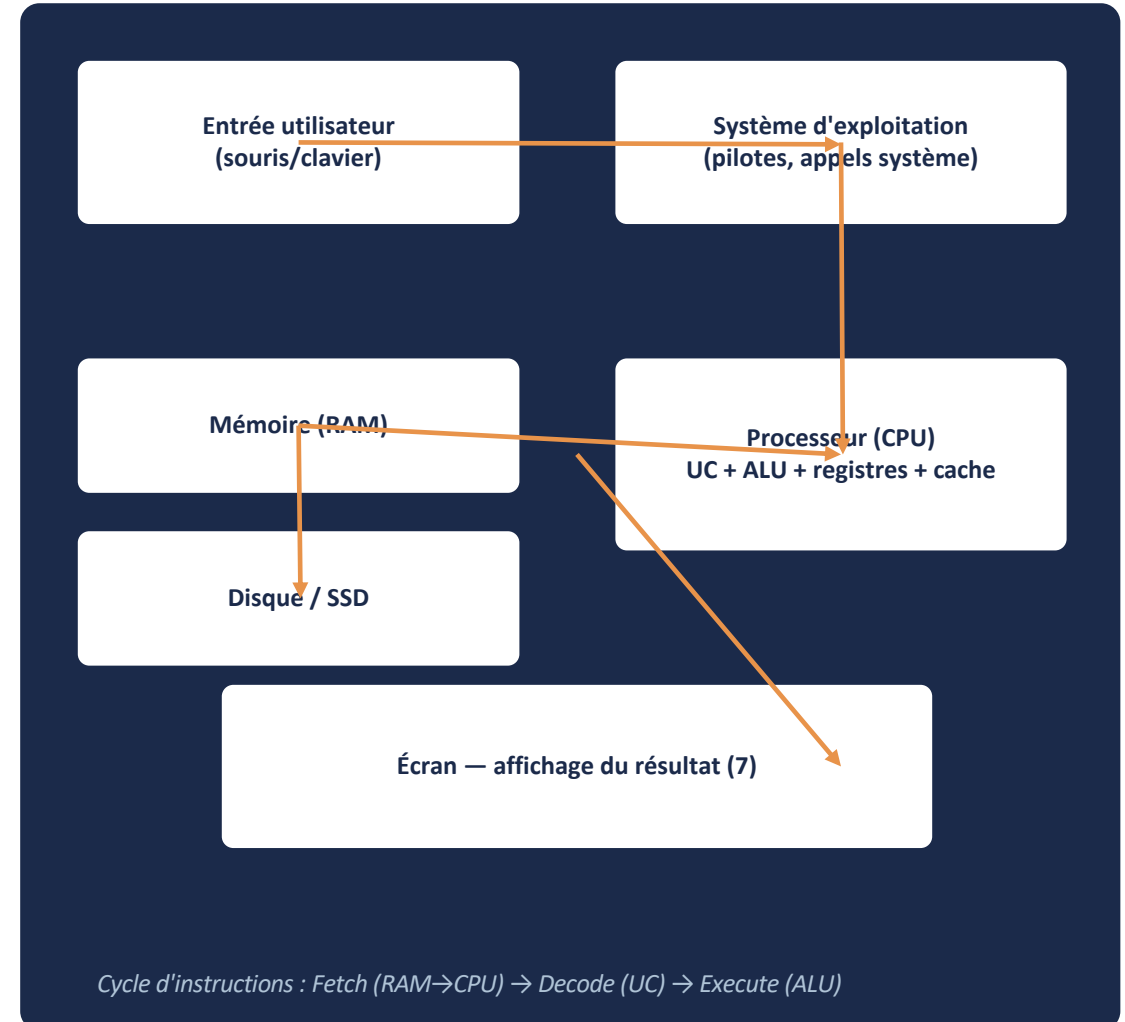
Processeur — Unité de contrôle : lit et décode les instructions, répartit les tâches entre l'ALU et les registres.

4

Processeur — ALU : effectue le calcul $2 + 5$.

5

L'OS utilise des appels système graphiques (ex. CreateWindow, DrawText) pour afficher le résultat ; les données passent par le bus vers la carte graphique / l'écran.





Quiz — Introduction aux systèmes d'exploitation

Q1

Vrai ou Faux

Le système d'exploitation (OS) sert uniquement à afficher une interface graphique pour l'utilisateur.

Q3

QCM

Quel registre contient l'instruction en cours d'exécution ?

- A. PC (Program Counter)
- B. IR (Instruction Register)
- C. ACC (Accumulateur)
- D. MDR (Memory Data Register)

Q2

Trouver la bonne définition

Associez chaque terme à sa définition :

1. Registres 2. ALU 3. Mémoire cache

- a) Réalise les calculs arithmétiques et logiques
- b) Stocke temporairement des données/instructions pour un traitement très rapide
- c) Mémoire très rapide qui conserve les données fréquemment utilisées



Quiz — Introduction aux systèmes d'exploitation (suite)

Q4**Complétez la phrase**

L'ALU du processeur met le résultat d'une addition dans le registre appelé _____.

Q6**QCM**

Quand l'utilisateur tape $2 + 5$ et appuie sur "=", dans quel composant matériel le calcul est-il effectué ?

A. Mémoire vive (RAM) B. Carte graphique C. ALU du processeur D. Registres

Q5**Reliez chaque appel système à sa fonction**

Appel système	Fonction
open()	1. Créer un nouveau processus
fork()	2. Lire ou ouvrir un fichier
write()	3. Écrire dans un fichier

Q7**Vrai ou Faux**

L'unité de contrôle (UC) est responsable de diriger l'exécution des instructions dans le processeur.



Rôles d'un système d'exploitation

Assurer le partage de la machine physique : CPU, RAM et entrées-sorties (E/S), entre toutes les applications actives.



Pour le CPU

Quel programme, parmi ceux présents en RAM, doit s'exécuter ? Le SE gère l'allocation du temps processeur aux différentes applications et processus en cours, assurant une utilisation efficace des ressources de traitement.



Pour la RAM

Comment allouer la mémoire aux différents programmes ? Comment sécuriser les données entre ces programmes ? Le SE gère l'allocation de la RAM, surveille son utilisation pour éviter les conflits et les surcharges.



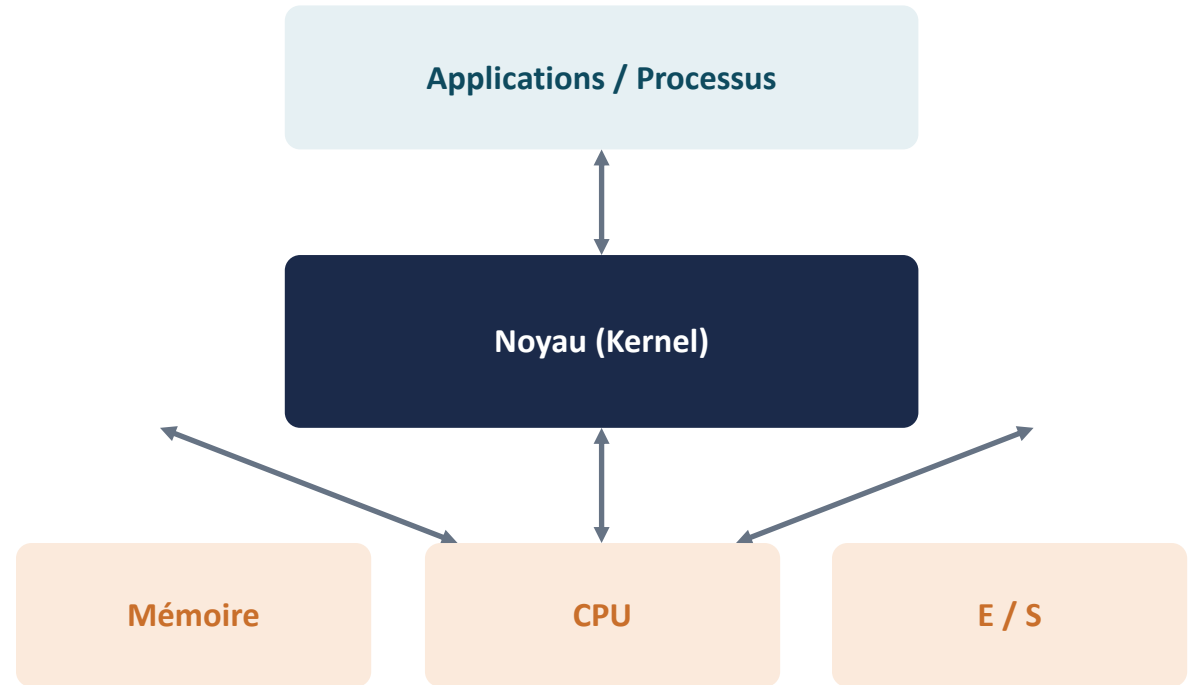
Pour les E/S

Dans quel ordre traiter les requêtes d'entrée-sortie (clavier, souris, disque, imprimante...) ? Le SE ordonnance ces requêtes afin d'optimiser les transferts et d'éviter les conflits d'accès.



Le noyau (kernel)

- Un système d'exploitation regroupe plusieurs composants : le **noyau**, l'**interpréteur de commandes** et le **système de fichiers**.
- Le noyau (en anglais kernel) est le composant principal d'un système d'exploitation.
- Le kernel (noyau) est essentiellement un ensemble de programmes/codes exécutables qui constituent le cœur du système d'exploitation.
- Il constitue l'interface centrale entre le matériel d'un ordinateur et ses processus.
- Son rôle est de faire communiquer le matériel et les processus, en gérant les ressources le plus efficacement possible.



Les applications ne parlent jamais directement au matériel : elles passent systématiquement par le noyau.



Le noyau : fonctions et architectures

De quoi est composé le kernel ? Le kernel contient différents modules de code, chacun ayant une fonction particulière.



Gestion de la mémoire

Gère la quantité de mémoire utilisée, l'endroit où elle est stockée, et son usage.



Gestion des processus

Détermine quels processus peuvent utiliser le CPU, à quel moment et pour combien de temps.



Pilotes de périphériques

Agit comme médiateur / interprète entre le matériel et les processus.



Appels système et sécurité

Reçoit les demandes de service des processus et contrôle les autorisations.

Deux grandes architectures de noyau

Monolithique

Prend en charge par lui-même la planification du CPU, la gestion de la mémoire, etc., sans déléguer ces tâches.

Modulaire

Peut étendre ses capacités grâce à des modules du noyau chargés dynamiquement (ex. : pilote ajouté à la volée).



Le système de fichiers

Le système de fichiers est un composant important d'un SE : il gère la manière dont les données sont stockées, organisées, récupérées et gérées sur un support de stockage (disque dur, SSD, etc.).

Nature d'un système de fichiers

Un système de fichiers est un ensemble de programmes et de routines, écrits en C ou C++, compilés en fichiers binaires exécutés par le noyau ou les programmes en espace utilisateur. Ces fichiers constituent la base du système de fichiers et permettent à l'ordinateur de gérer le stockage et l'organisation des données sur des dispositifs comme les disques durs.

Exemples de systèmes de fichiers

NTFS

New Technology File System — Windows

ext4

Fourth Extended File System — Linux

HFS+

Hierarchical File System Plus — macOS

FAT32

File Allocation Table 32-bit — compatibilité multi-systèmes



Système de fichiers : organisation et accès



Organisation des données

Les fichiers et répertoires sont organisés de manière hiérarchique, ce qui permet aux utilisateurs et aux programmes de trouver et gérer les données efficacement.



Gestion des accès

Le système de fichiers contrôle qui peut accéder à quels fichiers/répertoires, en appliquant des permissions de lecture, d'écriture et d'exécution. Exemple : NTFS utilise des ACL (Access Control Lists) pour définir ces permissions ; lors d'un accès, NTFS vérifie les droits configurés pour autoriser ou refuser l'opération.

Arborescence NTFS (Windows)

```
C:\
├── Program Files\
│   ├── Microsoft\
│   └── Adobe\
├── Users\
│   └── ouafae\
│       ├── Documents\
│       ├── Pictures\
│       └── Downloads\
└── Windows\
    ├── System32\
    └── Logs\
```

Exemple de structure hiérarchique d'un système de fichiers NTFS utilisé par Windows.



Système de fichiers : stockage et récupération

Stockage et récupération des données :

le système de fichiers gère la manière dont les données sont physiquement stockées sur le disque, et comment elles sont récupérées lorsqu'elles sont demandées.

Exemple NTFS — la Master File Table (MFT)

NTFS utilise la MFT (Master File Table), une base de données centralisée qui stocke des informations sur chaque fichier et répertoire présent sur un volume NTFS. Pour chaque fichier, la MFT peut notamment contenir son **nom**, sa **taille**, ses **dates de création** et de **modification**, ses **permissions**, ses attributs et l'emplacement des données sur le disque.

Fonctionnement

1

Lorsque vous créez un fichier ou un répertoire, NTFS ajoute une entrée correspondante dans la MFT.

2

Lorsque vous accédez à un fichier pour le récupérer, NTFS consulte la MFT pour retrouver les informations nécessaires : emplacement des données et attributs du fichier.



L'interpréteur de commandes

Interpréteur de commandes (shell) : un programme qui permet aux utilisateurs d'interagir avec le système d'exploitation en entrant des commandes textuelles.



Windows

L'interpréteur de commandes est `cmd.exe` (Invite de commandes) et PowerShell. Ils permettent de taper des commandes et des scripts pour gérer le système et les applications.



Unix / Linux

Les interpréteurs courants sont `bash` (Bourne Again Shell), `zsh` (Z Shell) et `sh` (Bourne Shell). Ils offrent des fonctionnalités similaires pour interagir avec le système et automatiser les tâches.



Les différents systèmes d'exploitation

Beaucoup d'OS existent : chaque type d'appareil demande un OS adapté à son architecture matérielle, plus ou moins performant et flexible selon sa conception.



Windows

Mac OS X
macOS

Linux



Android



iOS

	Windows	macOS	Linux	Android	iOS
Propriétaire	Microsoft	Apple Inc	Open source	Google LLC	Apple Inc
Ordinateur de bureau	✓		✓		
Ordinateur portable	✓		✓		
Mac (ordinateur Apple)	✓ (Boot Camp/VM)	✓	✓ (via VM)		
Tablette	✓ (selon modèle)		✓ (selon modèle)	✓	✓ (iPad)
Serveur	✓ (Windows Server)		✓		
Machine virtuelle (VM)	✓	✓	✓		
Appareil mobile			(versions spécifiques, ex. Ubuntu Touch)	✓	✓ (iPhone)



Évolution des versions de Windows

Année	Événement	Description
1974	CP/M	Système d'exploitation créé par Gary Kildall (Control Program for Microcomputers).
1980	QDOS	Développé par Tim Paterson, clone de CP/M (Quick and Dirty Operating System).
1981	MS-DOS	Microsoft reprend QDOS, le modifie et le lance pour les PC IBM sous le nom de MS-DOS.
1985	Windows 1.0	Premier OS Microsoft avec une interface graphique (GUI).
1995	Windows 95	Introduction du menu Démarrer et de la barre des tâches ; grande évolution de l'interface.
2001	Windows XP	Passage complet à Windows NT, fin du support de MS-DOS, stabilité améliorée.
2006	Windows Vista	Introduction de l'interface Aero et de la sécurité renforcée (UAC).
2009-2021	Windows 7 → 11	Progression continue des performances, de l'interface et des fonctionnalités.



WINDOWS

Les particularités essentielles de Windows



Interface graphique (GUI)

Interface conviviale avec des éléments comme le menu Démarrer et des fenêtres ajustables.



Multitâche

Gestion de plusieurs applications simultanément, avec des bureaux virtuels.



Sécurité

Fonctionnalités telles que Windows Defender et des mises à jour de sécurité régulières.



Compatibilité matérielle

Fonctionne sur une large gamme de matériel : ordinateurs de bureau, portables, tablettes.



Mises à jour

Modèle de mises à jour continues, avec améliorations et corrections régulières.



Commandes CMD : gestion des fichiers et répertoires

Commandes de gestion des répertoires

Commande	Description	Options	Syntaxe	Exemple
cd	Change le répertoire courant.	N/A	cd [chemin]	cd C:\Users\NomUtilisateur\Desktop
mkdir	Crée un nouveau répertoire.	N/A	mkdir [chemin\nom_du_répertoire]	mkdir C:\chemin\nouveauDossier
move	Déplace des fichiers ou répertoires vers un autre emplacement.	N/A	move [source] [destination]	move C:\source\MonFichier.txt C:\destination\
rename	Change le nom d'un fichier ou répertoire.	N/A	rename [ancien_nom] [nouveau_nom]	rename C:\fichier.txt nouveau_fichier.txt
robocopy	Copie des répertoires d'un emplacement à un autre.	/E : copie tous les sous-répertoires, y compris les vides.	robocopy [source] [dest] [fichiers] [options]	robocopy Desktop\rep1 Desktop\rep2 /E
rmdir	Supprime un répertoire et son contenu.	/s : supprime tout le contenu (sous-répertoires inclus). /q : mode silencieux.	rmdir /s /q [chemin_complet]	rmdir /s /q C:\chemin\répertoire



Commandes CMD : gestion des fichiers et répertoires

Affichage et création de contenu

Commande	Description	Explication des options	Syntaxe	Exemple
dir	Affiche la liste des fichiers et répertoires d'un répertoire.	/s : affiche aussi les fichiers des sous-répertoires.	dir [chemin\répertoire] /option	dir C:\chemin /b
echo	Affiche du texte, ou crée/écrit dans un fichier.	> : redirige la sortie en écrasant le contenu. >> : ajoute la sortie en fin de fichier.	echo [texte] > [chemin\Fichier.txt]	echo "Contenu du fichier" > C:\chemin\Fichier1.txt



Commandes CMD : gestion des fichiers et répertoires

Gestion des fichiers

Commande	Description	Explication des options	Syntaxe	Exemple
copy	Copie des fichiers d'un emplacement à un autre.	/y : pas de confirmation avant d'écraser. /v : vérifie l'écriture.	copy [source\file.txt] [destination\]	copy C:\source\file.txt C:\destination\
del	Supprime des fichiers.	/s : supprime aussi dans les sous-répertoires. /q : mode silencieux.	del [chemin\file.txt]	del C:\chemin\fichier.txt
type	Affiche le contenu d'un fichier texte.	N/A	type [nom_du_fichier.txt]	type C:\chemin\fichier.txt



Commandes PowerShell : manipulation de fichiers et répertoires

Commandes de création et de suppression

Commande	Description	Options	Exemple
New-Item	Crée un nouvel élément (fichier ou répertoire).	-Path : chemin de l'élément. -ItemType : File ou Directory.	New-Item -Path "C:\chemin\nouveauFichier.txt" -ItemType File
Remove-Item	Supprime un élément (fichier ou répertoire).	-Path -Recurse -Force	Remove-Item -Path "C:\chemin\fichier.txt"
Clear-Content	Efface le contenu d'un fichier sans le supprimer.	-Path : chemin du fichier.	Clear-Content -Path "C:\chemin\fichier.txt"
Add-Content	Ajoute plusieurs lignes de contenu.	-Path -Value	Add-Content -Path "..\fichier.txt" -Value @("Ligne 1","Ligne 2")



Commandes PowerShell : manipulation de fichiers et répertoires

Commandes de manipulation de fichiers et répertoires

Commande	Description	Options	Exemple
Copy-Item	Copie un élément vers un nouvel emplacement.	-Path -Destination -Recurse	Copy-Item -Path "C:\src.txt" -Destination "C:\dest\"
Move-Item	Déplace un élément vers un nouvel emplacement.	-Path -Destination -Force	Move-Item -Path "C:\src.txt" -Destination "C:\dest\"
Rename-Item	Renomme un élément.	-Path -NewName -Force	Rename-Item -Path "C:\fichier.txt" -NewName "nouveauNom.txt"
Get-Content	Récupère le contenu d'un élément.	-Path -TotalCount -Tail	Get-Content -Path "C:\chemin\fichier.txt"
Get-ChildItem	Affiche le contenu d'un répertoire.	-Path -Recurse -Filter	Get-ChildItem -Path "C:\chemin\"



Les variables d'environnement dans Windows

Définition : des paires clé-valeur utilisées par le système d'exploitation et les applications pour stocker des informations sur l'environnement d'exécution — chemins, configurations système, paramètres utilisateur.

Types de variables

- Système : applicables à tous les utilisateurs.
- Utilisateur : spécifiques à chaque utilisateur.

Exemples courants

PATH

Liste des répertoires où le système recherche les exécutables.

TEMP / TMP

Répertoire où les fichiers temporaires sont stockés.

USERNAME

Nom de l'utilisateur actuellement connecté.

USERPROFILE

Chemin du répertoire personnel de l'utilisateur actuellement connecté.



Ajouter une variable d'environnement

Avec CMD

```
REM --- Utilisateur ---  
setx MY_VARIABLE "C:\MonDossier"  
  
REM --- Système (droits admin requis) ---  
setx MY_VARIABLE "C:\MonDossier" /M
```

Avec PowerShell

```
# --- Utilisateur ---  
[Environment]::SetEnvironmentVariable(  
    "MY_VARIABLE", "C:\MonDossier", "User")  
  
# --- Système (droits admin requis) ---  
[Environment]::SetEnvironmentVariable(  
    "MY_VARIABLE", "C:\MonDossier", "Machine")
```

Portée « Utilisateur »

La variable n'est visible et accessible que par l'utilisateur qui l'a créée.
Aucun droit d'administrateur n'est nécessaire.

Portée « Système »

La variable est visible par tous les comptes utilisateurs de la machine. Elle nécessite des droits administrateur pour être créée ou modifiée.